

Tech Challenge – Fase 1

Sistema Integrado de Atendimento e Execução de Serviços – GarageHub

FIAP – Pós Tech em Software Architecture

Participante

Beatriz Tavernaro

RM: 371552

Turma: 17SOAT

Discord: Beatriz Tavernaro - 371552

1. Sobre o projeto

O GarageHub é um MVP de back-end desenvolvido para apoiar a gestão de uma oficina mecânica, centralizando processos que anteriormente poderiam depender de controles manuais ou planilhas.

A solução contempla o gerenciamento de clientes, veículos, serviços, peças e insumos, além do fluxo de criação de orçamentos e ordens de serviço.

O projeto foi desenvolvido aplicando conceitos de Domain-Driven Design (DDD), separação de responsabilidades e boas práticas de qualidade e segurança de software.

2. Tecnologias utilizadas

- .NET 10
- ASP.NET Core
- C#
- PostgreSQL
- Dapper
- Docker e Docker Compose
- JWT para autenticação
- Swagger / OpenAPI
- xUnit
- Moq
- FluentAssertions
- Coverlet
- SonarQube Cloud
- GitHub Actions

3. Documentação DDD

A documentação de Domain-Driven Design contém o Event Storming dos principais fluxos da aplicação, identificação dos elementos do domínio, diagramas desenvolvidos durante a modelagem e a Linguagem Ubíqua utilizada no projeto. Também é possível encontrar a documentação através do [README.md](#) do projeto

Link da documentação DDD:

<https://github.com/beatavernaro/GarageHub/tree/main/Documentacao/DDD>

4. Repositório

O código-fonte completo da aplicação, arquivos de infraestrutura, testes automatizados e documentação de execução estão disponíveis no repositório abaixo.

Link do repositório:

<https://github.com/beatavernaro/GarageHub>

5. Qualidade e testes

A aplicação possui testes automatizados para as regras de domínio, serviços da camada de aplicação e principais fluxos de integração.

Foram implementados testes unitários e testes de integração, incluindo cenários relacionados a autenticação, clientes, veículos, estoque, orçamento e geração de ordens de serviço.

A cobertura automatizada do projeto supera o requisito mínimo de 80% nos domínios críticos, sendo monitorada por meio do Coverlet e integrada à análise do SonarQube Cloud.

6. Análise de vulnerabilidades

A análise estática do código foi realizada utilizando o SonarQube Cloud, integrado ao repositório por meio do GitHub Actions.

O processo de análise considera aspectos de segurança, confiabilidade e manutenibilidade, além da cobertura dos testes automatizados.

6.1 Resultado da análise

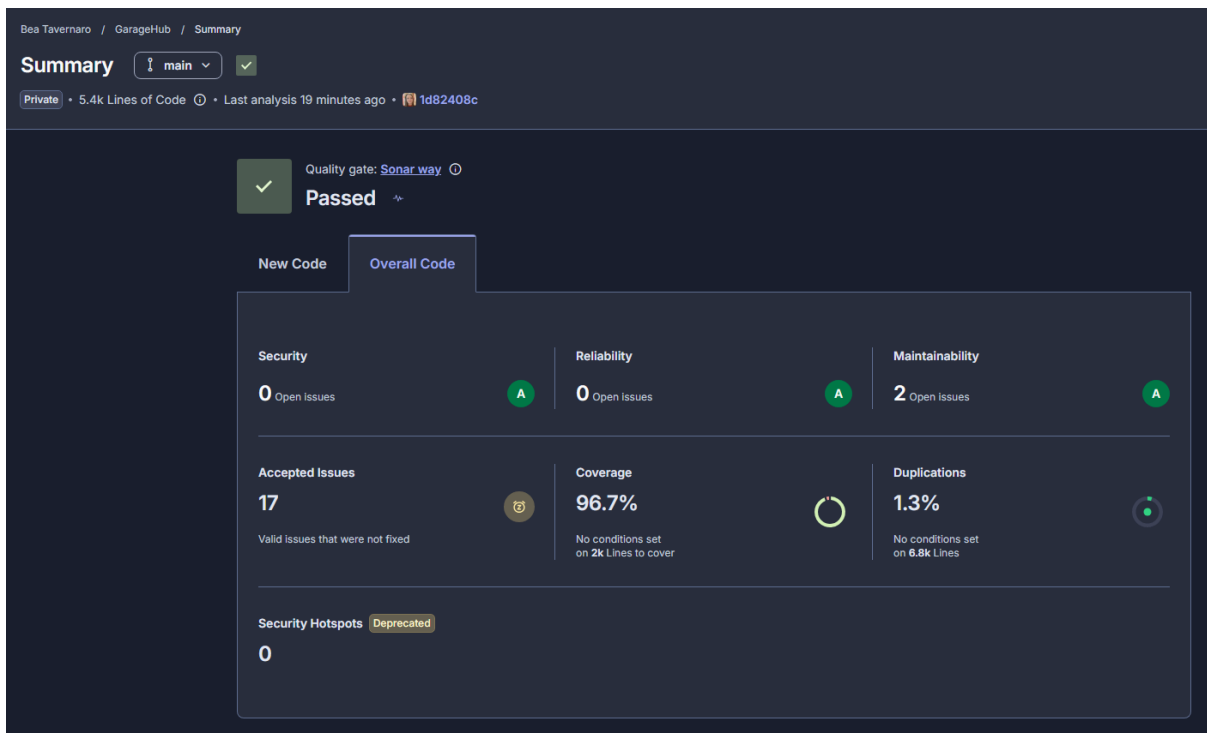


Figura 1 – Resultado geral da análise estática realizada pelo SonarQube Cloud. O projeto obteve Quality Gate aprovado, classificação A em segurança, confiabilidade e manutenibilidade, além de 96,7% de cobertura de testes.

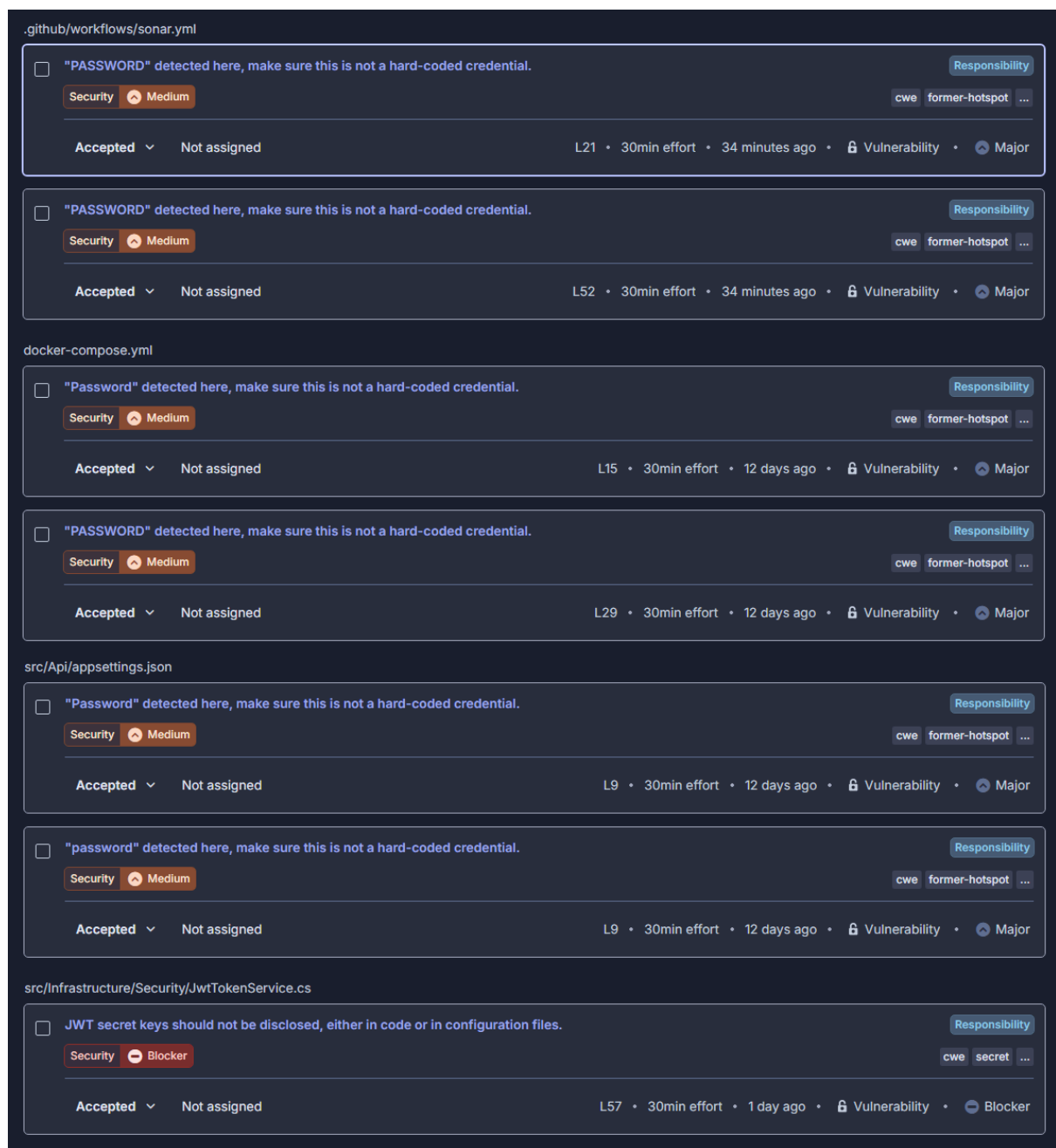


Figura 2 – Vulnerabilidades identificadas e classificadas como aceitas no SonarQube Cloud.

Os apontamentos apresentados na figura foram analisados individualmente e classificados como Accepted devido ao contexto acadêmico e controlado do projeto. As credenciais, senhas e chaves identificadas são utilizadas exclusivamente no ambiente de desenvolvimento e não fornecem acesso a sistemas, usuários ou dados reais.

A decisão de aceitar esses apontamentos foi tomada especificamente para este MVP acadêmico e não representa uma prática recomendada para ambientes de produção. Em um cenário de produção, senhas, chaves JWT, strings de conexão e demais secrets não devem ser armazenados diretamente no código-fonte ou em arquivos versionados no repositório. Esses valores deveriam ser fornecidos por mecanismos apropriados de

gerenciamento de secrets e variáveis de ambiente, com controle de acesso e possibilidade de rotação das credenciais.

Dessa forma, os riscos foram mantidos documentados no SonarQube Cloud de maneira consciente, demonstrando que os apontamentos foram avaliados e que as medidas necessárias para um ambiente produtivo são conhecidas.

6.2 Vulnerabilidades identificadas

Durante a análise estática realizada pelo SonarQube Cloud, foram identificados apontamentos relacionados à segurança da aplicação. Os principais casos analisados foram:

Execução do container Docker com usuário privilegiado

O SonarQube identificou que a imagem utilizada no estágio final do Dockerfile executava a aplicação utilizando o usuário padrão `root`. Essa configuração aumenta o impacto potencial de uma vulnerabilidade que permita acesso ao container.

Como correção, o Dockerfile foi alterado para executar a aplicação utilizando um usuário não privilegiado, reduzindo as permissões disponíveis dentro do container e seguindo o princípio do menor privilégio.

Credencial identificada no docker-compose

O SonarQube identificou a presença de uma configuração de senha no arquivo `docker-compose.yml` e sinalizou a possibilidade de uma credencial estar armazenada diretamente no código.

O apontamento foi analisado e classificado como risco aceito para o contexto do projeto, pois a credencial é utilizada exclusivamente no ambiente local de desenvolvimento, para inicialização do banco PostgreSQL via Docker Compose, não correspondendo a uma credencial de produção ou de um ambiente externo.

Hash BCrypt presente no arquivo de seed

Foi identificado pelo SonarQube um hash de senha BCrypt armazenado no arquivo `02-seed.sql`, utilizado para criação dos usuários iniciais do ambiente.

O apontamento foi analisado e mantido conscientemente, pois o hash pertence exclusivamente aos usuários fictícios utilizados no ambiente acadêmico e de desenvolvimento. Não se trata de uma credencial utilizada em produção ou associada a dados reais.

Resultado das correções de segurança

Os apontamentos encontrados foram individualmente analisados, permitindo diferenciar vulnerabilidades que exigiam alteração no código de situações relacionadas especificamente ao ambiente acadêmico e local. A execução privilegiada do container foi corrigida, enquanto os apontamentos referentes às credenciais de desenvolvimento foram avaliados e documentados como riscos aceitos devido ao contexto e à ausência de dados ou credenciais reais.

Code Smells e melhorias de qualidade

Além dos apontamentos diretamente relacionados à segurança, o SonarQube Cloud identificou alguns Code Smells, relacionados principalmente à manutenibilidade, legibilidade e confiabilidade do código.

Durante a análise, foram realizadas correções como:

- Melhoria no uso de operações assíncronas, utilizando corretamente métodos `async` e `await` e propagando `CancellationToken` quando aplicável.
- Redução de duplicação de código, como a substituição de mensagens repetidas por constantes.
- Simplificação de estruturas de código, incluindo loops que poderiam ser expressos de forma mais clara utilizando LINQ.
- Melhor tratamento de nulabilidade, removendo operadores `!` desnecessários e adequando trechos para tratar corretamente valores potencialmente nulos.
- Refatoração de propriedades, evitando lançamento de exceções diretamente em getters.
- Revisão de complexidade e quantidade de parâmetros, identificando métodos e construtores que poderiam dificultar a manutenção conforme a aplicação evolui.
- Melhorias nas consultas SQL, como explicitação da ordenação e simplificação de expressões booleanas. Alguns apontamentos específicos de SQL foram avaliados considerando que o projeto utiliza PostgreSQL e determinadas regras detectadas pelo SonarQube eram direcionadas a PL/SQL/Oracle.

Esses apontamentos não representam necessariamente vulnerabilidades exploráveis, mas indicam oportunidades de melhorar a qualidade interna do software. As correções realizadas contribuíram para tornar o código mais legível, previsível e fácil de manter, complementando a análise de segurança realizada no projeto.

6.3 Conclusão da análise

Após a análise e o tratamento dos apontamentos identificados pelo SonarQube Cloud, o projeto obteve Quality Gate aprovado, com classificação A em Segurança, Confiabilidade e Manutenibilidade.

Ao final da análise, o projeto apresentou 0 issues de segurança abertas, 0 Security Hotspots, 96,7% de cobertura de testes e 1,3% de duplicação de código.

Os apontamentos mantidos como aceitos foram avaliados individualmente e correspondem a decisões tomadas considerando o contexto acadêmico do projeto. Em um ambiente produtivo, especialmente os casos relacionados ao gerenciamento de credenciais e secrets, seriam necessárias medidas adicionais de segurança.